
Cognitive Kernel v1.6

Quantum Adversarial Extension Specification

Project:	Wolf-CoG-OS / AAIS Federated Cognitive Architecture	Document Status:	DRAFT-STABLE — Implementation Candidate
Supersedes:	Cognitive Kernel v1.5	Version Date:	June 2026
Extension Set:	QIGEM (Quantum-Resistant IGEM) ZKALS (Zero-Knowledge ASIL Lineage System) dAPID (Distributed APID Mesh)		
Compatibility:	Fully backward-compatible with CK v1.5 governance stack		
Audience:	Senior Systems Architects Cryptographers AI Governance Engineers		
Prepared By:	jon Wolf-CoG-OS Architecture Division Grayling Charter Township, MI, United States		
Timestamp:	Monday, 15 June 2026 — 01:34 EDT		

Table of Contents

- §0 Preface & Design Rationale
 - 0.1 Quantum Decryption Threat Vector
 - 0.2 Identity Proof Without Exposure
 - 0.3 Centralized Injection Detection Failure Mode
 - 0.4 Version Delta Table (v1.5 → v1.6)
- §1 Updated Governance Stack Notation (v1.6)
 - 1.1 Full Governance Stack with v1.6 Substrate Annotations
 - 1.2 Core Compatibility Rule (LEGACY_MODE)
- §2 QIGEM: Quantum-Resistant Identity Graph Encryption Model

- 2.1 Threat Model Addressed
- 2.2 Algorithm Substitution Table (Formal)
- 2.3 QIGEM Key Architecture
- 2.4 Quantum Threat Detection Protocol
- 2.5 QIGEM Hybrid Transition Protocol (Epoch-Based)
- 2.6 Graph Traversal Under QIGEM

§3 ZKALS: Zero-Knowledge ASIL Lineage System

- 3.1 Problem Statement
- 3.2 Zero-Knowledge Proof Model
- 3.3 ZKALS Proof Generation Flow
- 3.4 ZKALS Verification Protocol
- 3.5 ZKALS Integration Points
- 3.6 Proof Scope Selection Rules
- 3.7 ZKALS + QIGEM Interaction

§4 dAPID: Distributed APID Mesh

- 4.1 Problem Statement
- 4.2 Design Principle: Mesh Consensus Over Singleton Verdicts
- 4.3 Byzantine Fault Tolerance Model
- 4.4 dAPID Evaluation Protocol (Round-Based)
- 4.5 Blind Commit Protocol (Anti-Coordination Defense)
- 4.6 Consensus Disposition Rules
- 4.7 Dissent Analysis and FTSS Integration
- 4.8 dAPID Mesh Recomposition
- 4.9 Legacy APID Interoperability (v1.5 Nodes)

§5 Cross-Layer Integration Protocol (v1.6)

- 5.1 New Event Types (Internal Event Bus Additions)
- 5.2 Critical Event Chains (v1.6 Additions)
- 5.3 v1.6 Consensus Contract (Updated)

§6 Security Threat Model (v1.6 Extended)

§7 Performance & Complexity Analysis

- 7.1 Computational Cost Analysis
- 7.2 Latency Mitigation Strategies

§8 Redundancy Architecture (v1.6 Extended)

§9 Migration Guide: CK v1.5 → v1.6

§10 One-Line System Definition (v1.6)

§11 Roadmap: CK v1.7 Extensions (Preview)

Section 0 — Preface & Design Rationale

Cognitive Kernel v1.6 is a formally specified substrate upgrade to the CK v1.5 governance architecture, targeting three distinct threat vectors that were explicitly out of scope for v1.5's design envelope. CK v1.6 introduces no changes to governance authority, precedence ordering, or decision logic within the existing layer stack. All modifications are confined to cryptographic substrates, lineage verification protocols, and injection detection consensus mechanisms. The governance hierarchy is preserved with exact fidelity.

This specification is authoritative for all implementation candidates operating within the Wolf-CoG-OS / AAIS Federated Cognitive Architecture. All references to v1.5 components retain their original definitions unless explicitly superseded herein.

0.1 — Quantum Decryption Threat Vector

Classical **IGEM** (Identity Graph Encryption Model) employs AES-256-GCM for symmetric encryption and relies implicitly on RSA or ECC primitives for key exchange. Against a cryptographically relevant quantum computer (CRQC), this substrate presents two distinct vulnerabilities. First, Grover's algorithm provides a quadratic speedup over brute-force search, reducing the effective security of AES-256 from 256 bits to approximately 128 bits — insufficient against a well-resourced nation-state quantum adversary. Second, Shor's algorithm enables polynomial-time factorization and discrete logarithm computation, rendering RSA and ECC key exchange mechanisms entirely broken under CRQC-capable conditions.

QIGEM (Quantum-Resistant Identity Graph Encryption Model) replaces both the symmetric key exchange substrate and the digital signature scheme with NIST Post-Quantum Cryptography (PQC) Round 4 standardized algorithms, specifically those of the CRYSTALS family (Kyber and Dilithium), while retaining AES-256-GCM as the

symmetric cipher within a mandatory hybrid configuration during the transition epoch.

0.2 — Identity Proof Without Exposure

ASIL lineage verification in CK v1.5 requires that `AuthorRecord` contents — including `author_id`, `intent_signature`, `canonical_hash`, and `derivation_policy` — be presented in plaintext to all nodes participating in a Consensus Engine merge validation round. This design exposes authorship metadata to any federation node that participates in consensus, including those operating at PROVISIONAL trust tier, which by definition have not achieved full trust validation.

This plaintext exposure creates a tractable attack surface for ARPL (Author Record Poisoning and Lineage) attacks, including attribution stripping, structural cloning, and targeted derivation chain manipulation. **ZKALS** (Zero-Knowledge ASIL Lineage System) eliminates this exposure entirely. Under ZKALS, any verifying node can confirm the cryptographic validity of an author's lineage claim — including authorship, derivative classification, intent binding, and ASIL revocation status — without receiving any `AuthorRecord` plaintext. Provable authorship is maintained; author identity is never disclosed.

0.3 — Centralized Injection Detection Failure Mode

APID (Adversarial Prompt Injection Detection) in CK v1.5 is implemented as a per-node singleton pipeline. Each node independently evaluates inputs through its local five-stage detection pipeline and emits a self-signed `APIDReport`. The Consensus Engine consumes these reports as authoritative. This design is structurally sound against a single compromised node; however, it provides no defense against a coordinated multi-node collusion attack.

A set of N compromised or colluding federation nodes can coordinate to emit uniformly fraudulent PASS dispositions for inputs that a correctly functioning pipeline would classify as REJECT or ESCALATE_TO_ASIL. The Consensus Engine, receiving $N \times \text{PASS}$ reports with no cross-node validation mechanism, has no basis for detecting the fraud and will accept the falsified consensus. **dAPID** (Distributed APID Mesh) replaces the singleton report model with a Byzantine-fault-tolerant distributed consensus protocol, cryptographically enforcing that no coalition of fewer than one-third of mesh participants can forge a PASS disposition.

0.4 — Version Delta Table (v1.5 → v1.6)

IGEM	AES-256-GCM + RSA/ECC key exchange	Replaced by QIGEM: ML-KEM-1024 + ML-DSA-5 + SHA3-512 hybrid suite	Substrate Replacement	Quantum key extraction (Grover/Shor)
ASIL Lineage Verification	Plaintext AuthorRecord comparison during consensus	Replaced by ZKALS: zk-STARK zero-knowledge proof exchange	Protocol Replacement	Author identity exposure; ARPL attacks
APID	Per-node singleton pipeline; self-signed APIDReport	Replaced by dAPID: BFT distributed mesh consensus with blind commit	Architecture Replacement	Coordinated APID falsification (colluding nodes)
ASIL	Unchanged	Now custodian of QIGEM keys; countersigns ZKALSProof envelopes; arbitrates dAPID escalation	Role Extension (no precedence change)	N/A — authority layer unchanged
FTSS	Fed by APIDReport + AuthorRecord data	Now fed by ZKALSVerificationResult + dAPIDRoundResult; LEGACY_MODE penalty applied	Input Source Update	Trust score accuracy under new protocols
Consensus Engine	5-condition contract; consumes per-node APIDReports	8-condition contract; consumes dAPIDRoundResult + ZKALSProof; QIGEM epoch	Contract Extension	Consensus integrity under quantum/ZK/BFT model

		check added		
CIEMS	Unchanged	Unchanged; evaluates impact of QIGEM epoch transitions and dAPID escalations on constraint state	No Change	N/A
RPDS	Unchanged	Unchanged; receives stronger threat signal from dAPID mesh and ZKALS nonce replay detection	Signal Quality Improvement (no architectural change)	N/A
CK Core	Unchanged	Unchanged	No Change	N/A
Internal Event Bus	Original canonical event set	Five new event types added: QuantumThreatAlert, ZKALSProofResult, dAPIDRoundResult, MeshRecompositionEvent, EpochAdvancementEvent	Additive Extension	Cross-layer coordination for v1.6 protocols

Section 1 — Updated Governance Stack Notation (v1.6)

The CK v1.6 governance precedence hierarchy is strictly identical to CK v1.5. No layer has been added, removed, promoted, or demoted. The annotations below reflect

substrate modifications and overlay protocol integrations only. Precedence ordering is authoritative and non-negotiable.

ARCHITECTURAL INVARIANT

The governance precedence hierarchy of CK v1.6 is identical to CK v1.5 in all respects. QIGEM, ZKALS, and dAPID are substrate replacements and overlay protocols — they do not constitute new governance layers and carry no independent precedence authority. Any implementation that interprets v1.6 extension modules as governance layers is non-conformant.

1.1 — Full Governance Stack with v1.6 Substrate Annotations

1	ASIL	UNCHANGED — Authority Layer	Lineage now proven via ZKALS (zero-knowledge proof); ASIL remains sole custodian of QIGEM key lifecycle; ASIL countersigns all federation-wide QuantumThreatAlerts and ZKALSProof envelopes
2	IGEM → QIGEM	SUBSTRATE REPLACED — Post-Quantum Cryptography	Classical AES-256-GCM/RSA/ECC substrate replaced with ML-KEM-1024 + ML-DSA-5 + SHA3-512 hybrid suite; three-epoch transition model (EPOCH_0 / EPOCH_1 / EPOCH_2); all traversal tokens replaced with

			QTraversalToken
3	CIEMS	UNCHANGED — Constraint Evaluation Layer	Evaluates constraint impact of QIGEM epoch transitions; approves cross- epoch merge operations; evaluates dAPID ESCALATE_TO_ASI L events for constraint state implications
4	APID → dAPID	DETECTION REPLACED — Distributed Mesh Consensus	Singleton per-node detection pipeline replaced with BFT distributed mesh; blind commit protocol defeats coordinated falsification; minimum production mesh: 7 nodes; all verdicts now dAPIDRoundResult (not per-node APIDReport)
5	RPDS	UNCHANGED — Behavioral Monitoring	Receives materially stronger threat signal from dAPID mesh dissent analysis and ZKALS NONCE_REPLAYED detection; no architectural changes; benefits passively from v1.6 enhancements
6	FTSS	UNCHANGED — Trust Scoring	Input sources updated: ZKALSVerification Result replaces raw AuthorRecord

			for identity_consistency_score; dAPIDRoundResult replaces APIDReport for behavioral scoring; LEGACY_MODE nodes receive temporal_reliability_score penalty of -0.10; ZKALS_NONCE_REPLAYED triggers immediate score drop
7	Consensus Engine	UNCHANGED — Merge Decision Authority	Consensus Contract extended from 5 to 8 conditions; now consumes dAPIDRoundResult (not per-node APIDReport); ZKALSProof verified=VALID required for all contributing authors; QIGEM epoch compatibility check added as Condition 8
8	CK Core	UNCHANGED — Base Kernel	No modifications. All v1.6 extensions operate above or as overlays to CK Core.

1.2 — Core Compatibility Rule (LEGACY_MODE)

Any node running CK v1.5 that connects to a v1.6 federation is automatically classified as operating in LEGACY_MODE. The following constraints apply to all LEGACY_MODE nodes without exception:

- LEGACY_MODE nodes receive a reduced FTSS `temporal_reliability_score` of `-0.10`, applied persistently until the node upgrades to v1.6.
 - LEGACY_MODE nodes cannot participate in ZKALS proof exchange rounds. They are excluded from ZKALSProof generation and verification roles.
 - LEGACY_MODE nodes may participate in dAPID evaluation rounds only as LEGACY_EVALUATOR entities, subject to the constraints defined in Section 4.9.
 - LEGACY_MODE nodes are capped at **PROVISIONAL** trust tier maximum, regardless of prior trust history. Tier elevation requires full v1.6 upgrade and re-evaluation.
 - LEGACY_MODE nodes remain valid federation members and may contribute outputs to the Consensus Engine subject to PROVISIONAL-tier rules. Their participation is not suspended — it is constrained.
-

Section 2 — QIGEM: Quantum-Resistant Identity Graph Encryption Model

2.1 — Threat Model Addressed

Classical IGEM employs AES-256-GCM for symmetric encryption of identity graph nodes and edges, and relies on RSA-4096 or ECDH for key exchange and ECDSA-P384 for digital signatures. The security of these primitives rests on the computational hardness of integer factorization and the elliptic curve discrete logarithm problem — both of which are efficiently solvable by a sufficiently capable quantum computer running Shor's algorithm.

Additionally, Grover's algorithm provides a quadratic speedup for unstructured search, reducing AES-256's effective quantum security from 256 bits to approximately 128 bits. While 128-bit post-quantum security may be acceptable for some applications, the sensitivity of IGEM identity graph data — containing authorship records, derivation lineages, and canonical hashes for the full federation — necessitates the higher security margin provided by full post-quantum primitives.

THREAT CLASSIFICATION: CRQC

A Cryptographically Relevant Quantum Computer (CRQC) is defined as a quantum computing system capable of executing Shor's algorithm on RSA-4096 or ECDH-384 key sizes within operationally relevant timeframes. Current NIST guidance (2024) anticipates CRQC capability becoming a realistic adversarial tool within the 10–15 year horizon. Harvest-now-decrypt-later (HNDL) attacks mean that data encrypted today under classical primitives may be retroactively decrypted once CRQC capability matures. QIGEM addresses both the present HNDL threat and the future real-time CRQC threat.

QIGEM replaces the classical encryption substrate with NIST PQC Round 4 standardized algorithms, maintaining a mandatory hybrid classical + post-quantum configuration during the federation transition epoch to ensure no degradation of security during migration.

2.2 — Algorithm Substitution Table (Formal)

Function	CK v1.5 Algorithm	CK v1.6 Algorithm	Security Level	NIST Standard Reference
Symmetric Encryption	AES-256-GCM	AES-256-GCM + CRYSTALS-Kyber-1024 (hybrid mode)	256-bit post-quantum	FIPS 203 (ML-KEM)
Key Encapsulation	RSA-4096 / ECDH	CRYSTALS-Kyber-1024 (ML-KEM)	NIST Level 5	FIPS 203
Digital Signatures	ECDSA-P384	CRYSTALS-Dilithium-5 (ML-DSA)	NIST Level 5	FIPS 204
Hash Function	SHA-256	SHA-3-512 (Keccak)	256-bit quantum-resistant	FIPS 202
Graph Edge MAC	HMAC-SHA-256	HMAC-SHA3-512	256-bit quantum-resistant	FIPS 202
Key Derivation	HKDF-SHA256	HKDF-SHA3-	256-bit	SP 800-56C

Function	CK v1.5 Algorithm	CK v1.6 Algorithm	Security Level	NIST Standard Reference
		512	quantum-resistant	

NOTE: HYBRID MODE MANDATE

Hybrid mode (classical primitive + post-quantum primitive operating in parallel) is **mandatory** during the transition epoch (EPOCH_1). A session key is derived from the combination of both the classical and post-quantum shared secrets using HKDF-SHA3-512. Security is maintained as long as at least one primitive remains unbroken. Pure post-quantum mode (EPOCH_2) activates only after all nodes in a federation have individually confirmed QIGEM full readiness via a countersigned ASIL broadcast.

2.3 — QIGEM Key Architecture

The `QIGEMKeyRecord` supersedes the `v1.5 IGENKeyRecord` for all federation nodes operating in `EPOCH_1` or `EPOCH_2`. ASIL remains the sole custodian of key lifecycle operations — generation, rotation, and revocation — with no change to this authority relationship. The schema is formally defined as follows:

```
QIGEMKeyRecord {  key_id:                string,  key_epoch:
int,  algorithm_suite:    HYBRID_PQC | PURE_PQC |
LEGACY_CLASSICAL,      // — Key Encapsulation
_____  kem_algorithm:                ML-KEM-
1024,  kem_public_key:    bytes,          // Kyber-1024 public key
kem_encapsulated_secret: bytes,          // Encapsulated symmetric
key    // — Digital Signature _____
sig_algorithm:    ML-DSA-5,  sig_public_key:    bytes,
// Dilithium-5 verification key    // — Symmetric Layer
_____  sym_algorithm:                AES-
256-GCM,  sym_key_hash:    SHA3-512,          // Hash of symmetric
key – never stored plaintext    // — Lifecycle
_____  rotation_policy:
EPOCH_BASED | EVENT_TRIGGERED | QUANTUM_THREAT_ALERT,
rotation_epoch_interval: int,          // Number of consensus cycles per
rotation  revocation_triggers: [    RPDS_HARD_RESET,    ASIL_ESCALATION,
QUANTUM_THREAT_DETECTED,    MANUAL    ],  asil_custodian:    true
// ASIL remains sole key custodian – non-negotiable }
```

2.4 — Quantum Threat Detection Protocol

QIGEM introduces a new Internal Event Bus event type: `QuantumThreatAlert`. This event is emitted by the QIGEM subsystem under three distinct triggering conditions: (1) key usage anomalies consistent with brute-force quantum probing patterns are detected in the cryptographic telemetry stream; (2) external threat intelligence broadcast at the federation level signals confirmed or probable CRQC activity; or (3) RPDS detects cryptographic integrity failures whose statistical signature is inconsistent with classical attack patterns.

```
QuantumThreatAlert {  alert_id:          string,    threat_confidence:
float (0.0 - 1.0),    affected_key_ids:  string[],    recommended_action:
ROTATE_KEYS | EMERGENCY_REKEY | SUSPEND_FEDERATION,    broadcast_scope:
LOCAL | FEDERATION_WIDE,    timestamp:          ISO8601,
asil_countersigned:  bool                // ASIL must countersign for
FEDERATION_WIDE broadcast                // LOCAL
alerts do not require countersignature }
```

A `threat_confidence` value at or above 0.70 triggers mandatory ASIL evaluation for countersignature of a federation-wide broadcast. ASIL's countersignature decision is final and not subject to override by any other governance layer. Upon federation-wide broadcast confirmation, all nodes initiate key rotation per their configured `rotation_policy`.

2.5 — QIGEM Hybrid Transition Protocol (Epoch-Based)

The QIGEM transition is structured as three formally defined epochs. No node may skip an epoch in sequence; advancement is gated by verifiable federation-wide conditions confirmed through ASIL-countersigned broadcast.

Epoch	Designation	Cryptographic Mode	Node Scope	FTSS Penalty	Advancement Gate
EPOCH_0	LEGACY	Classical IGEM only (AES-256-GCM / RSA / ECC)	v1.5 nodes; pre-migration v1.6 nodes	−0.10 on temporal_reliability_score	QIGEM deployment package installed and key record generated
EPOCH_1	HYBRID	CRYSTALS-Kyber-1024 + AES-256-GCM dual-layer; ML-	All v1.6 nodes (default operational epoch)	None	CIEMS stability confirmed; ASIL-countersign

Section 3 — ZKALS: Zero-Knowledge ASIL Lineage System

3.1 — Problem Statement

In CK v1.5, Consensus Engine merge validation requires participating nodes to present the contents of the relevant `AuthorRecord` in plaintext. The fields exposed during this process include: `author_id`, `intent_signature`, `canonical_hash`, and `derivation_policy`. This plaintext presentation is visible to every node participating in the consensus round — including nodes operating at PROVISIONAL trust tier, which may be operated by parties not fully trusted within the federation.

The consequences of this design are threefold. First, author identity is exposed to potentially adversarial or untrusted participants. Second, the lineage map — which encodes structural relationships between outputs and derivation chains — is leaked in a form that enables targeted ARPL attacks, including attribution stripping (severing an output from its legitimate author) and structural cloning (replicating a lineage structure with substituted authorship). Third, in federated deployments where node operators are organizational adversaries, this design constitutes a systematic privacy violation incompatible with multi-party trust models.

ZKALS (Zero-Knowledge ASIL Lineage System) eliminates all three consequences. Any verifying node can confirm the full validity of an author's lineage claim — authorship, derivative classification, intent binding, derivation chain integrity, and ASIL revocation status — without receiving any `AuthorRecord` plaintext. The proof is mathematically binding; the prover cannot construct a valid proof without possessing the actual `AuthorRecord` as a cryptographic witness.

3.2 — Zero-Knowledge Proof Model

ZKALS implements Non-Interactive Zero-Knowledge Proofs (NIZKPs) using zk-STARKs (Scalable Transparent Arguments of Knowledge). The selection of zk-STARKs over zk-SNARKs (Succinct Non-Interactive Arguments of Knowledge) is a deliberate architectural decision driven by three considerations critical to the CK v1.6 security model:

- **No trusted setup required:** zk-SNARKs require a setup ceremony that produces a structured reference string; compromise of the setup process introduces a backdoor exploitable by anyone who participated. zk-STARKs require no such ceremony and have no trusted setup vulnerability.
- **Post-quantum security:** zk-SNARKs rely on pairing-based elliptic curve cryptography, which is broken by Shor's algorithm. zk-STARKs are hash-based and are considered post-quantum secure under standard assumptions about the quantum hardness of the underlying hash function (SHA3-512).
- **Transparent verifiability:** zk-STARK verifiers require no secret information and can be executed by any node independently without coordination with the proof generator.

```
ZKALSProofSystem {  proof_scheme:  zk-STARK,  hash_function:  SHA3-512,
// Quantum-resistant hash for Merkle commitments  field:          Binary
extension field GF(2^256),  // — Statements proven (without revealing
witness) ————— statements: [  "I am the legitimate author of
this content",  "This output is a valid DIRECT | DERIVATIVE | EXTENSION |
INFERENCE",  "My derivation chain has not been tampered with",  "My
intent_signature matches the canonical_hash of this output",  "My
AuthorRecord has not been revoked by ASIL"  ],  // — Secrets never
revealed ————— secrets: [  author_id,
canonical_hash,  intent_signature_contents,  derivation_chain_nodes  ]
}
```

3.3 — ZKALS Proof Generation Flow

Proof generation is initiated by the content-owning node in response to a `ZKALSProofRequest` from any authorized verifying entity. The ASIL-issued nonce is single-use and round-bound; proof reuse across rounds or contexts is cryptographically detectable and constitutes a `NONCE_REPLAYED` violation.

```
ZKALSProofRequest {  content_hash:          SHA3-512,  // Hash of
content being validated (PUBLIC)  derivative_class:
DIRECT_REPRODUCTION  | TRANSFORMATION
| EXTENSION  | INDEPENDENT_INFERENCE,
proof_scope:          AUTHORSHIP | LINEAGE | INTENT | FULL,
requesting_node:      string,  requesting_node_trust_tier: TrustTier
// From FTSS – determines required scope } ZKALSProof {  proof_id:
string,  content_hash:      string,  // PUBLIC – what is
being proven about  derivative_class:      string,  // PUBLIC –
the claimed classification  proof_scope:      string,  //
PUBLIC – what the proof covers  stark_proof_bytes:
bytes,  // The zk-STARK proof – all secrets within
verification_key_hash:      SHA3-512,  // Hash of public verification key
only  proof_generation_timestamp: ISO8601,  proof_expiry:
ISO8601,  // Proofs expire – replay of old proofs is invalid
```

```
asil_nonce:                string,           // ASIL-issued single-use nonce,
bound to proof  asil_countersignature:      ML-DSA-5      // ASIL signs
proof envelope (not contents) }
```

3.4 — ZKALS Verification Protocol

Any node holding the public verification key may execute ZKALS proof verification independently. Verification requires no communication with the proof generator, no access to `AuthorRecord` contents, and no coordination with ASIL beyond the initial countersignature check against the ASIL public key. The result is a fully auditable `ZKALSVerificationResult` record.

```
ZKALSVerificationResult {  proof_id:                string,
verification_status:      VALID                     | INVALID
| EXPIRED                  | NONCE_REPLAYED
| ASIL_REVOKED,  verified_statements:      string[], // Which
statements were proven true  content_hash_match:      bool,
derivative_class_valid:      bool,  expiry_valid:      bool,
nonce_fresh:                bool,  asil_countersignature_valid:  bool,
verifying_node:             string,  verification_timestamp:
ISO8601 }
```

Verification failure handling is formally specified as follows:

- **INVALID:** Emit `IdentityDelta` event to ASIL immediately; flag source node with `LINEAGE_POISONING` threat class in the next dAPID evaluation round.
- **EXPIRED:** Request a fresh proof from the generating node; do not apply FTSS penalty. Proof expiry is a normal operational condition.
- **NONCE_REPLAYED:** Immediate ASIL escalation; apply `identity_consistency_score` drop to source node FTSS record; trigger RPDS TRIGGERED mode for source node; reject all pending Consensus Engine merges involving output from that source node.
- **ASIL_REVOKED:** Full lineage seal for all output associated with source node; suspend source node from federation participation pending ASIL adjudication.

3.5 — ZKALS Integration Points

ZKALS replaces plaintext `AuthorRecord` presentation at all four points within the CK v1.6 system where author identity was previously disclosed:

1. **Consensus Engine merge validation** (PRIMARY integration): All merge operations now require a valid `ZKALSProof` for every contributing author. This is encoded as Condition 7 of the v1.6 Consensus Contract (see Section 5.3).
2. **FTSS identity_consistency_score updates**: The score is now computed from `ZKALSVerificationResult` records rather than raw `AuthorRecord` comparisons. Verification status outcomes map to score deltas defined in the FTSS scoring specification.
3. **Cross-Node Reflection Mirror divergence checks**: Lineage divergence analysis now operates on proof verification outcomes rather than raw lineage data comparisons, eliminating the need to exchange lineage plaintext across nodes.
4. **ARPL export authorization**: `ZKALSProof` (AUTHORSHIP scope or higher) replaces `AuthorRecord` disclosure as the authorization credential for ARPL export approval, preventing author identity exposure during the export workflow.

3.6 — Proof Scope Selection Rules

The required proof scope is determined by the trust tier of the requesting node. Higher-trust requestors require less proof coverage because their trust tier provides an independent verification signal. Lower-trust requestors require maximum coverage to compensate for the reduced independent verification confidence their tier represents.

```
ProofScopeSelectorRule {    requesting_tier: SOVEREIGN    → scope: AUTHORSHIP
// Minimum sufficient; SOVEREIGN nodes have maximum
// independent trust signal    requesting_tier: VERIFIED    → scope:
AUTHORSHIP | LINEAGE    // Lineage
verification required for VERIFIED tier    requesting_tier: PROVISIONAL →
scope: FULL    // Maximum verification
required;    // PROVISIONAL tier has minimum
trust signal    requesting_tier: QUARANTINED → proof_exchange: SUSPENDED
// No proof exchange; ASIL direct arbitration only    requesting_tier:
EXPULSED    → proof_exchange:
PROHIBITED    // No proof exchange; lineage
sealed }
```

3.7 — ZKALS + QIGEM Interaction

All `ZKALSProof` envelopes are transmitted under QIGEM `EPOCH_1` or `EPOCH_2` encryption. The `stark_proof_bytes` field is subject to double encryption: it is first encrypted under the content author's QIGEM identity key (ML-KEM-1024 encapsulated, AES-256-GCM encrypted), and then encrypted again under the transport session key established for the current peer connection. This double-encryption ensures that compromise of the transport layer does not expose the proof contents.

ASIL countersignatures on `ZKALSProof` envelopes use ML-DSA-5 (Dilithium-5), ensuring that the signature binding is quantum-resistant by design, consistent with the QIGEM substrate. Proof envelopes signed under ECDSA-P384 (v1.5 era) are not accepted in v1.6 federation consensus rounds.

Section 4 — dAPID: Distributed APID Mesh

4.1 — Problem Statement

CK v1.5 APID is implemented as a per-node singleton: each node independently executes the five-stage Adversarial Prompt Injection Detection pipeline on each input and emits a self-signed `APIDReport` containing its local disposition. The Consensus Engine treats each `APIDReport` as authoritative for the node that issued it.

CRITICAL FAILURE MODE: COLLUDING NODE ATTACK

If N nodes in a federation are compromised and coordinate their behavior, they can collectively emit fraudulent PASS dispositions for any input — including inputs that a correctly functioning pipeline would classify as REJECT or ESCALATE_TO_ASIL. The Consensus Engine receives $N \times \text{PASS}$ reports. Because per-node APID provides no cross-node validation mechanism, the engine has no basis for detecting the fraud and will accept the falsified consensus disposition. Per-node APID is structurally incapable of defending against coordinated report

falsification by a colluding minority of nodes.

dAPID (Distributed APID Mesh) is the architectural response to this failure mode. It replaces the singleton report model with a Byzantine-fault-tolerant distributed consensus protocol in which no single node's verdict is trusted in isolation, and in which a colluding coalition of fewer than one-third of mesh participants is mathematically incapable of forging a PASS disposition.

4.2 — Design Principle: Mesh Consensus Over Singleton Verdicts

The central design principle of **dAPID** is that the final injection detection disposition is the result of a cryptographically validated consensus among a quorum of independent APID evaluators — the APID Mesh — rather than any individual node's assessment. No single node's dAPID verdict has operative effect; only the consensus verdict is accepted by the Consensus Engine.

```
APIDMeshTopology {  mesh_type:          FULLY_CONNECTED | RING |
RANDOM_SPARSE,      minimum_quorum:      ceil(2/3 * active_mesh_nodes),
// Byzantine fault tolerance: requires  $f < n/3$   evaluation_round_timeout:
milliseconds,      mesh_composition_policy:  RANDOM_SELECTION
| FTSS_WEIGHTED    | GEOGRAPHIC_DIVERSE,
mesh_refresh_interval:  int      // Consensus cycles between mesh
recomposition        // Default: 50 }
```

4.3 — Byzantine Fault Tolerance Model

dAPID implements a Byzantine Fault Tolerant (BFT) consensus model. The fundamental BFT tolerance bound is: the system tolerates up to $f_{\text{Byzantine}}$ (malicious or faulty) nodes where $f < n/3$, and n is the total number of mesh participants in the current round. The following table enumerates the fault tolerance profile at key mesh sizes:

Mesh Size (n)	Max Byzantine Faults Tolerated (f)	Minimum Quorum Required	Deployment Classification
3	0	2	Not recommended for production; minimum viable mesh only
4	1	3	Development /

Mesh Size (n)	Max Byzantine Faults Tolerated (f)	Minimum Quorum Required	Deployment Classification
			test environments only
7	2	5	Minimum production mesh — tolerates 2 colluding adversaries
10	3	7	Standard production mesh
13	4	9	Recommended production mesh — tolerates 4 colluding adversaries
19	6	13	High-security federation deployments

4.4 — dAPID Evaluation Protocol (Round-Based)

Each **dAPID** evaluation proceeds through four sequential phases within a single round. All mesh participants evaluate the identical input, identified only by its SHA3-512 hash, ensuring that no participant can evaluate a modified or substituted version. The full round schema is defined below:

```
dAPIDEvaluationRound {  round_id:          string,  input_hash:
SHA3-512,              // Hash of input being evaluated
// All nodes evaluate the same input  mesh_participants:
string[],              // Node IDs selected for this round  round_start:
ISO8601,  round_timeout:  ISO8601,          // — Phase 1: Independent
Evaluation _____  // No inter-node communication
permitted during Phase 1  per_node_verdicts: { [node_id: string]: {
local_threat_class:  ThreatClass[],          local_threat_score:  float,
local_disposition:  PASS | QUARANTINE | REJECT | ESCALATE_TO_ASIL,
verdict_signature:  ML-DSA-5                // Node signs its own verdict  } },
// — Phase 2: Blind Commit _____  // Nodes
commit to verdict hash before Phase 3 reveal  verdict_commits:
{ [node_id: string]: SHA3-512                // SHA3-512(verdict || node_id ||
round_id || nonce) },          // — Phase 3: Reveal + Consensus
_____  // Verdicts revealed simultaneously after
all commits received  revealed_verdicts: { [node_id: string]: verdict
// Revealed after all Phase 2 commits received },          // — Phase 4:
Consensus Result _____  consensus_disposition:
PASS | QUARANTINE | REJECT | ESCALATE_TO_ASIL,  quorum_reached:
bool,  quorum_count:      int,  dissenting_nodes:  string[],
// Nodes whose verdict differed from consensus  consensus_signature:  ML-
DSA-5[]  // All quorum participants sign final verdict }
```

4.5 — Blind Commit Protocol (Anti-Coordination Defense)

The four-phase blind commit protocol is the primary defense mechanism against the specific failure mode that motivated dAPID's design: colluding nodes observing the honest nodes' verdicts and adjusting their own to manufacture a false consensus. By requiring cryptographic commitment before reveal, the protocol ensures that no node can see any other node's verdict before it has irrevocably committed to its own.

5. **Phase 1 — EVALUATE:** Each mesh node independently executes the complete five-stage APID pipeline on the input identified by `input_hash`. Zero inter-node communication is permitted during this phase. Any detected communication attempt during Phase 1 is treated as a coordination violation and triggers FTSS audit for the communicating nodes.
6. **Phase 2 — COMMIT:** Each node broadcasts a cryptographic commitment of the form: `SHA3-512(verdict || node_id || round_id || nonce)`. The commitment reveals nothing about the verdict content. Nodes that fail to submit a commit within the round timeout are treated as abstentions for Phase 3.
7. **Phase 3 — REVEAL:** After all commits are received (or after the reveal window opens following commit timeout), all nodes simultaneously reveal their verdicts. Late reveals are treated as abstentions and do not count toward quorum. The commitment hash is verified for each reveal; any mismatch between the committed hash and the revealed verdict constitutes a binding fraud event and triggers immediate ASIL escalation for the offending node.
8. **Phase 4 — CONSENSUS:** BFT aggregation of all verified revealed verdicts. The consensus disposition is computed according to the rules in Section 4.6. All quorum participants sign the final consensus verdict with ML-DSA-5.

4.6 — Consensus Disposition Rules

The following rules govern consensus disposition computation from the set of revealed verdicts. Rules are evaluated in priority order; the first matching condition determines the disposition.

```

ConsensusDispositionRule {    // — Unanimous consensus
    if unanimous(quorum):
disposition = unanimous_verdict    // — Supermajority (≥ 2/3 of mesh)
    if supermajority(REJECT | ESCALATE_TO_ASIL):
disposition = strictest_majority_verdict    // Strictest:
ESCALATE_TO_ASIL > REJECT > QUARANTINE > PASS    if supermajority(PASS |
QUARANTINE):    disposition = majority_verdict    // — Simple majority (≥
1/2 but < 2/3)    if simple_majority(REJECT |
ESCALATE_TO_ASIL):    disposition = QUARANTINE    // Escalate
to caution; not full reject —    // insufficient supermajority
for REJECT    if simple_majority(PASS):    disposition = PASS,
dissent_flag = true    // Flagged for FTSS review; not rejected    // —
Quorum failure (timeout or insufficient reveals)    if
quorum_failed:    disposition = QUARANTINE    // Default safe state on quorum
failure    // — Systematic dissent pattern
    if dissent_pattern_detected:
disposition = ESCALATE_TO_ASIL,    ftss_audit_trigger = true    // Possible
coordinated falsification }

```

4.7 — Dissent Analysis and FTSS Integration

Dissenting nodes — those whose locally-computed verdict diverged from the final consensus disposition — are subject to a structured review and scoring process. This process is designed to detect both isolated judgment failures and coordinated falsification patterns.

- Immediate FTSS `behavioral_consistency_score` adjustment is applied to each dissenting node. The delta is proportional to the severity of the disagreement (PASS vs. REJECT is a maximum-severity disagreement; QUARANTINE vs. PASS is a moderate-severity disagreement).
- Pattern tracking is maintained per node across rounds. A node that registers 3 consecutive dissenting verdicts relative to the consensus triggers RPDS TRIGGERED mode for that node.
- Coordinated dissent detection: if 2 or more nodes share identical non-consensus verdicts across multiple rounds — particularly if those nodes have other behavioral correlations — this is treated as evidence of probable coordination. A coordinated dissent detection event triggers a full FTSS audit for all identified nodes and an ASIL notification.

4.8 — dAPID Mesh Recomposition

Static mesh membership over extended periods creates the risk of long-running collusion becoming established within a stable group. **dAPID** mitigates this risk through periodic and event-driven mesh recomposition.

- **Scheduled recomposition:** The mesh is recomposed randomly every N consensus cycles, where N is configurable per federation (default: 50 cycles). Random recomposition prevents any colluding group from relying on stable co-membership across rounds.
- **Forced recomposition:** Any ESCALATE_TO_ASIL consensus disposition triggers an immediate forced mesh recomposition before the next evaluation round.
- **FTSS-weighted selection:** Mesh composition is biased toward SOVEREIGN and VERIFIED tier nodes. PROVISIONAL tier nodes are limited to a maximum of 20% of any mesh composition. QUARANTINED and EXPELLED tier nodes are excluded from mesh participation entirely.

4.9 — Legacy APID Interoperability (v1.5 Nodes)

CK v1.5 nodes operating singleton APID pipelines may participate in **dAPID** rounds with the following constraints. Their participation maintains federation connectivity while the trust penalty structure incentivizes migration.

- v1.5 nodes are designated `LEGACY_EVALUATOR` for the purposes of dAPID round participation.
- `LEGACY_EVALUATOR` verdicts are included in mesh evaluation but are assigned a trust weight of **0.5x** (compared to 1.0x for fully dAPID-capable nodes). This reduced weight is applied before BFT aggregation.
- `LEGACY_EVALUATOR` nodes cannot participate in Phase 2 (blind commit), as they lack the cryptographic infrastructure to generate commitment hashes bound to round-specific nonces. Their verdicts are submitted directly in Phase 3 (reveal). This structural limitation means their verdicts could theoretically

be adjusted after observing Phase 2 commits from dAPID-capable nodes; the 0.5× weight discount accounts for this reduced assurance.

- FTSS applies a `temporal_reliability_score` penalty of -0.05 for `LEGACY_EVALUATOR` status, applied cumulatively with the -0.10 `LEGACY_MODE` penalty for a total of -0.15 per affected score dimension.

Section 5 — Cross-Layer Integration Protocol (v1.6)

5.1 — New Event Types (Internal Event Bus Additions)

CK v1.6 adds five new canonical event types to the Internal Event Bus. These events integrate the three new extension modules into the existing cross-layer communication fabric. All existing v1.5 event types remain unchanged and continue to be emitted by their original sources.

Event Type	Emitting Source	Primary Consumers	Description
<code>QuantumThreatAlert</code>	QIGEM subsystem	CIEMS, ASIL, RPDS, all nodes (federation-wide)	Emitted on cryptographic anomaly detection consistent with quantum probing or CRQC activity
<code>ZKALSProofResult</code>	ZKALS verification module (any verifying node)	FTSS, Consensus Engine, ASIL (on failure)	Emitted after every proof verification; contains <code>ZKALSVerificationResult</code> ; replaces <code>AuthorRecord</code> event in v1.5
<code>dAPIDRoundResult</code>	dAPID consensus aggregator	Consensus Engine, FTSS, RPDS	Emitted after every completed dAPID evaluation round; contains consensus disposition and dissent data; replaces

Event Type	Emitting Source	Primary Consumers	Description
			APIDReport in v1.5
MeshRecompositionEvent	dAPID mesh manager	FTSS, ASIL (notification), all mesh participants	Emitted whenever dAPID mesh membership changes; contains new member list and recomposition trigger reason
EpochAdvancementEvent	QIGEM epoch controller	All nodes, CIEMS, ASIL (countersignature required)	Emitted by QIGEM when transitioning between cryptographic epochs; requires ASIL countersignature before operative effect

5.2 — Critical Event Chains (v1.6 Additions)

The following event chains formally define the system response to the four primary v1.6 operational scenarios. These chains supplement — and do not replace — the event chains defined in CK v1.5.

Chain E: Quantum Threat Detected

QIGEM detects cryptographic anomaly → emits `QuantumThreatAlert` (threat_confidence, broadcast_scope=LOCAL) to Internal Event Bus → CIEMS evaluates constraint impact of key rotation sequence → if `threat_confidence ≥ 0.70`: ASIL evaluates alert → ASIL countersigns `QuantumThreatAlert` for `FEDERATION_WIDE` broadcast → all federation nodes receive countersigned alert → all nodes initiate key rotation to current epoch's key suite → RPDS activates `MODE_TRIGGERED` across federation → FTSS recalculates trust scores incorporating new cryptographic integrity signal → if `recommended_action = EMERGENCY_REKEY`: ASIL initiates expedited rotation sequence bypassing normal epoch cycle → if `recommended_action = SUSPEND_FEDERATION`: ASIL locks consensus state pending threat assessment completion.

Chain F: ZK Proof Fails (NONCE_REPLAYED)

ZKALS verification returns `NONCE_REPLAYED` for source node S → emits `IdentityDelta` to ASIL immediately → ASIL escalates: classifies source node S as active lineage integrity threat → emits `ZKALSProofResult` (status=`NONCE_REPLAYED`) to Internal Event Bus → dAPID flags node S as `LINEAGE_POISONING` ThreatClass in next evaluation round → FTSS decrements `identity_consistency_score` for node S by delta proportional to severity → RPDS activates `TRIGGERED` mode for node S → Consensus Engine emits rejection for all pending merge operations that include output contributions from node S → if second `NONCE_REPLAYED` within 10 consensus cycles: ASIL initiates full lineage seal for node S and suspends from federation.

Chain G: dAPID Mesh Detects Coordinated Falsification

Dissent pattern analysis in `dAPIDRoundResult` confirms coordinated falsification signature across identified node set C → dAPID consensus aggregator emits `ESCALATE_TO_ASIL` disposition → ASIL receives escalation → ASIL locks consensus state (no new merges approved) → FTSS initiates full audit of all nodes in set C: `behavioral_consistency_score` and `identity_consistency_score` evaluated across historical round data → CIEMS re-evaluates all pending consensus states that involved input from nodes in set C → ASIL downgrades all confirmed colluding nodes in set C to `QUARANTINED` tier → dAPID mesh forced recomposition excluding all `QUARANTINED` nodes → ASIL issues `PROVISIONALLY_QUARANTINED` notice to node operators → CIEMS releases consensus lock upon confirming clean state baseline.

Chain H: New Node Requests v1.6 Federation Admission

Node N submits admission request → QIGEM validates submitted `QIGEMKeyRecord` for algorithm suite compatibility and epoch compatibility → ZKALS issues `AUTHORSHIP`-scope proof challenge to node N → node N generates and returns `ZKALSProof` (scope=`AUTHORSHIP`) → ZKALS verifies proof: if `INVALID` or `EXPIRED`, admission denied → dAPID includes node N as observer-only (no verdict contribution) for 3 consecutive evaluation rounds — evaluation integrity check period → FTSS assigns initial `PROVISIONAL` trust score based on proof outcome and observation period behavioral data → if all checks pass: CIEMS validates constraint compatibility of new node's declared operating parameters → ASIL reviews and countersigns admission authorization → node N admitted to federation at `PROVISIONAL` tier → `MeshRecompositionEvent` emitted to incorporate node N into eligible mesh pool (capped at 20% `PROVISIONAL` representation).

5.3 — v1.6 Consensus Contract (Updated)

The CK v1.5 Consensus Contract specified five binary conditions, all of which must be simultaneously satisfied for a merge operation to be approved by the Consensus Engine. CK v1.6 extends this contract with three additional conditions. The binary validity rule is unchanged: any single failure among all eight conditions constitutes an invalid consensus. There are no partial-validity states.

#	Condition	Source	v1.6 Change
1	ASIL lineage valid for all contributing authors	ASIL	Unchanged from v1.5
2	IGEM/QIGEM integrity confirmed — no graph node tampering detected	QIGEM	Unchanged from v1.5 (QIGEM provides equivalent assurance)
3	CIEMS constraint evaluation: STABLE or RESOLVED	CIEMS	Unchanged from v1.5
4	FTSS trust score $\geq$ threshold for all participating nodes	FTSS	Unchanged from v1.5; threshold now incorporates ZKALS and dAPID score inputs
5	RPDS mode: PASSIVE or WATCHFUL (not TRIGGERED or HARD_RESET)	RPDS	Unchanged from v1.5
6	dAPID consensus disposition = PASS or resolved QUARANTINE	dAPID	NEW (v1.6) — replaces per-node APIDReport PASS requirement
7	ZKALSProof verified = VALID for all contributing authors	ZKALS	NEW (v1.6) — replaces plaintext AuthorRecord validation
8	QIGEM epoch	QIGEM / CIEMS	NEW (v1.6) —

#	Condition	Source	v1.6 Change
	compatibility confirmed for all participating nodes (no cross-epoch merge without CIEMS approval)		prevents cross-epoch merge integrity failures

Section 6 — Security Threat Model (v1.6 Extended)

The CK v1.5 security threat model enumerated 12 threat categories. CK v1.6 extends this model with 6 additional threat categories specific to the quantum, zero-knowledge, and distributed consensus attack surfaces introduced by the QIGEM, ZKALS, and dAPID extension set. The table below presents the 6 v1.6-specific threats. The original v1.5 threat entries are unchanged and remain authoritative.

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
T-13	Quantum Key Extraction Attack	Adversary deploys CRQC to decrypt intercepted identity graph traffic or archived ciphertext encrypted under classical RSA/ECC key exchange	QIGEM (primary); ASIL (key custody)	QIGEM QuantumThreatAlert on anomalous key usage patterns; RPDS cryptographic integrity failure signature analysis; external threat intelligence broadcast	Chain E (Quantum Threat Detected): ASIL countersigns federation-wide alert; all nodes rotate to EPOCH_1 key suite; RPDS activates TRIGGERED ; EPOCH_2 advancement initiated if all nodes confirm readiness

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
T-14	ZK Proof Forgery	Adversary attempts to construct a stark_proof_bytes payload that passes ZKALS verification without possessing the legitimate AuthorRecord as the cryptographic witness; may be combined with stolen verification key hash	ZKALS (primary); ASIL (nonce issuance and countersignature)	zk-STARK proof verification failure (INVALID status); mismatch between claimed content_hash and verifiable proof commitment; ASIL nonce not present or not matching issued record	Immediate INVALID disposition; emit IdentityDelta to ASIL; LINEAGE_POISONING flag in next dAPID round; FTSS score drop; RPDS TRIGGERED for source node; all pending merges involving source node rejected
T-15	Mesh Collusion (Coordinated APID Falsification)	A coalition of $\geq f$ nodes ($f < n/3$) compromised by a common adversary coordinate to submit uniform false PASS verdicts for a malicious input that should be REJECTED or ESCALATED; relies on inter-node pre-coordination before the dAPID round	dAPID (primary); FTSS (dissent tracking); RPDS (behavioral pattern)	Dissent pattern analysis across multiple rounds identifies statistically improbable verdict correlation among identified node set; BFT quorum threshold prevents success with $f < n/3$; Phase 2 blind commit prevents mid-round coordination	Chain G (Coordinated Falsification): ESCALATE_TO_ASIL; ASIL locks state; FTSS audit; CIEMS re-evaluation; confirmed colluding nodes downgraded to QUARANTINED; forced mesh recomposition

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
				adjustment	
T-16	Blind Commit Timing Attack	Adversary monitors Phase 2 commit submission timing to infer verdict content (e.g., a node that commits very quickly may have a predetermined verdict); uses timing inference to adjust Phase 3 reveal before the reveal window closes	dAPID blind commit protocol; QIGEM encrypted transport	Commit submission timing anomaly detection; statistical analysis of timing correlation between commit timestamps and verdict outcomes across multiple rounds; RPDS behavioral monitoring	Commit windows are fixed-duration (not reveal-on-all-submitted) to eliminate timing inference; QIGEM encrypted transport obscures commit content; late reveals treated as abstentions ; timing-correlated nodes flagged for FTSS behavioral review
T-17	Epoch Rollback Attack	Adversary forces a node or set of nodes to revert from EPOCH_1 (HYBRID) or EPOCH_2 (PURE_PQC) to EPOCH_0 (LEGACY classical), thereby restoring the classical cryptographic attack surface and enabling	QIGEM (epoch enforcement); ASIL (countersignature requirement); CIEMS	Unauthorized epoch downgrade attempt detected by QIGEM epoch controller; any epoch transition without valid ASIL-countersigned EpochAdvancementEvent rejected; RPDS detects classical	Epoch rollback without valid ASIL-countersigned EpochAdvancementEvent is unconditionally rejected; attempting node flagged with LINEAGE_POISONING in dAPID; FTSS score drop; ASIL

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
		quantum key extraction against the rolled-back node's traffic		algorithm negotiation from a node declared as EPOCH_1+	notification; CIEMS evaluates constraint impact; node may be suspended pending investigation
T-18	ZK Nonce Replay (Cross-Round)	Adversary captures a previously valid ZKALSProof from an earlier consensus round and attempts to present it in a new round context to assert authorship of content produced by a different node or at a different time; exploits the time window before proof expiry	ZKALS (primary — nonce binding, proof expiry); ASIL (nonce registry)	ASIL-maintained nonce registry detects reuse of a nonce previously consumed in a prior round; proof_expiry timestamp check; round_id binding in nonce construction prevents cross-round reuse; ZKALS returns NONCE_REPLAYED status	Chain F (NONCE_REPLAYED): immediate ASIL escalation; IdentityDelta emitted; FTSS identity_consistency_score drop; RPDS TRIGGERED for source node; all pending merges involving source node rejected; second offense within 10 cycles triggers full lineage seal

Section 7 — Performance & Complexity Analysis

CK v1.6's architectural extensions introduce measurable computational overhead relative to v1.5. This section provides a formal cost analysis and the mitigation strategies that reduce the net operational impact to acceptable levels. All overhead figures are approximations based on published NIST PQC benchmark data and known zk-STARK complexity results; implementation-specific measurements must be performed in target hardware environments.

7.1 — Computational Cost Analysis

Operation	v1.5 Cost / Algorithm	v1.6 Cost / Algorithm	Overhead Delta	Justification
Key generation	RSA-4096 generation (~600ms typical)	ML-KEM-1024 (Kyber) generation (~420ms typical)	−30% (faster)	Kyber key generation is computationally cheaper than RSA at equivalent or superior security levels
Encryption per identity node	AES-256-GCM only	AES-256-GCM + Kyber KEM encapsulation (hybrid)	+15%	Hybrid layer adds one KEM encapsulation operation per session establishment ; not per-message
Signature generation	ECDSA-P384 (~2ms)	ML-DSA-5 / Dilithium-5 (~3ms)	+50%	Dilithium signatures are larger (4,595 bytes vs. 96 bytes for ECDSA-P384) and generation is proportionally costlier; parallelizable
Signature verification	ECDSA-P384 (~3ms)	ML-DSA-5 / Dilithium-5 (~4.2ms)	+40%	Larger signature and public key size increases verification time; acceptable

Operation	v1.5 Cost / Algorithm	v1.6 Cost / Algorithm	Overhead Delta	Justification
				given the security margin gained
Lineage validation	AuthorRecord plaintext comparison $O(1)$	zk-STARK proof verification $O(\log n)$ where n = proof depth	+variable	Depends on lineage chain depth; shallow proofs (depth ≤ 4) are comparable to plaintext comparison; deep chains (depth > 10) impose material overhead. Proof caching mitigates repeated verification costs.
Injection detection (per input)	Single-node APID pipeline (1 execution, 1 round)	dAPID mesh: N independent executions + 4-phase consensus protocol	+$N \times$latency (offset by parallelism)	N mesh node evaluations run in parallel during Phase 1; total wall-clock time is single-node latency + communication overhead (Phase 2-4), not $N \times$ single-node latency
Consensus round total overhead	v1.5 standard 5-condition contract	v1.5 baseline + ZKALSPProof verification + dAPIDRoundR result aggregation	+variable	Significantly offset by: (a) ZKALSPProof caching (not computed per-request), (b) dAPID Phase 1 parallelization, (c) SOVEREIGN-tier minimum-scope proof optimization

7.2 — Latency Mitigation Strategies

The following strategies are mandatory implementation requirements for CK v1.6 conformant deployments. They collectively reduce the net latency impact of v1.6 extensions to operationally acceptable levels.

- **dAPID Phase 1 Parallelization:** All N mesh node evaluations execute concurrently during Phase 1. The wall-clock latency of Phase 1 is approximately equal to the slowest single-node evaluation plus communication propagation time — not the sum of all evaluations. Phase 2 (commit), Phase 3 (reveal), and Phase 4 (consensus) are also parallelized across mesh participants.
- **ZKALSProof Pre-generation and Caching:** ZKALSProofs are generated proactively by each node and cached until expiry. Proof generation is not triggered by incoming verification requests; instead, the node serves pre-computed proofs from its local cache. Cache refresh is triggered by approaching expiry or ASIL nonce renewal, not by consensus round cadence.
- **QIGEM Hybrid Mode Parallel Operation:** During EPOCH_1, the classical and post-quantum KEM operations execute in parallel at session establishment. The combined session key is derived from both outputs via HKDF-SHA3-512. This eliminates sequential overhead and ensures that hybrid mode introduces no additional latency relative to running either primitive alone.
- **ZKALS Proof Scope Optimization:** SOVEREIGN-tier nodes exchange AUTHORSHIP-scope proofs only (minimum proof size and verification cost). The proof scope escalation by tier (Section 3.6) ensures that the highest-trust, highest-throughput nodes operate at minimum cryptographic overhead while lower-trust nodes bear proportionally higher verification burden.
- **ML-KEM Key Reuse Window:** Kyber KEM session keys are valid for a configurable number of consensus cycles (default: 10), avoiding per-round KEM encapsulation overhead for established peer connections. Key reuse

windows are bounded by the `rotation_epoch_interval` in the `QIGEMKeyRecord` and do not violate the key rotation policy.

Section 8 — Redundancy Architecture (v1.6 Extended)

The CK v1.5 redundancy architecture defined a 5-axis model providing layered defense against single-point failures across the governance stack. CK v1.6 extends this model with two additional axes introduced by the QIGEM and ZKALS extension modules. The 5 original axes are unchanged and remain authoritative.

Axis	Redundancy Type	Mechanism	v1.6 Status	Failure Survived
1	Governance Layer Redundancy	Multi-layer precedence stack; each layer independently validates before passing to next	Unchanged from v1.5	Compromise or failure of any single governance layer
2	Cryptographic Identity Redundancy	IGEM/QIGEM identity graph provides tamper-evident record of all node relationships and lineage chains	Enhanced: QIGEM post-quantum substrate strengthens integrity guarantees	Identity spoofing; graph tampering; unauthorized lineage claim
3	Behavioral Monitoring Redundancy	RPDS + FTSS provide independent behavioral and trust-score signals; each operates without requiring the other	Unchanged; signal quality improved by v1.6 inputs	Single monitoring system failure; manipulated behavioral data in one channel

Axis	Redundancy Type	Mechanism	v1.6 Status	Failure Survived
4	Consensus Validation Redundancy	All 8 Consensus Contract conditions must pass simultaneously; any single failure blocks the merge	Extended from 5 to 8 conditions in v1.6	Partial compromise of consensus inputs; manipulation of any single validation channel
5	Federation Node Redundancy	Federated architecture ensures no single node is a single point of failure for any governance function	Enhanced: dAPID BFT model formalizes $f < n/3$ node failure tolerance	Node failure; node compromise; network partition affecting minority of nodes
6	Cryptographic Redundancy	QIGEM EPOCH_1 mandatory hybrid mode: classical AES-256-GCM + post-quantum Kyber KEM operate simultaneously; session key derived from both. Compromise of one primitive does not compromise the session key if the other remains secure.	NEW (v1.6)	Complete classical algorithm compromise (e.g., RSA broken by CRQC); complete post-quantum algorithm compromise (e.g., implementation flaw in Kyber); any single-primitive failure during transition epoch
7	Proof Redundancy	ZKALS zk-STARK proofs are independently verifiable	NEW (v1.6)	Prover node offline or unreachable during verification;

Axis	Redundancy Type	Mechanism	v1.6 Status	Failure Survived
		by any federation node without coordination with the original prover and without requiring the prover to be online or reachable. Verification requires only the public verification key and the proof bytes.		prover node compromised after proof generation; man-in-the-middle interference with proof transmission (proof binding remains valid)

NOTE: 7-AXIS REDUNDANCY SUMMARY

CK v1.6 implements a 7-axis redundancy model. Each axis addresses a distinct failure surface. No two axes share a single point of failure. The system is designed such that any 3 axes could simultaneously degrade without producing a complete governance failure, provided the Consensus Engine retains access to at least one valid signal per Consensus Contract condition.

Section 9 — Migration Guide: CK v1.5 → v1.6

The following 5-phase migration sequence (plus one future phase) provides the formally specified path from a CK v1.5 federation to full v1.6 operation. Each phase must be completed and validated before the subsequent phase begins. Phase

completion criteria are formally gated; no phase may be skipped or compressed without ASIL authorization.

Phase	Name	Actions	Completion Criteria	Risk Level
Phase 0	Assessment	Audit all nodes for QIGEM algorithm compatibility (Kyber/Dilithium hardware/software support). Identify all v1.5 LEGACY_MODE nodes and enumerate their migration dependencies. Establish baseline FTSS scores for all nodes prior to migration (pre-migration baseline for comparison post-migration). Document mesh topology candidates for initial dAPID composition.	Complete inventory of all federation nodes with migration readiness classification. Baseline FTSS snapshot recorded. No nodes begin QIGEM deployment until Phase 0 audit complete.	Low
Phase 1	QIGEM EPOCH_1 Deployment	Deploy QIGEM package to all v1.6-capable nodes. Generate QIGEMKeyRecord entries (algorithm_suite=HYBRID_PQC) for each node under	100% of v1.6-capable nodes operating in EPOCH_1 (HYBRID) mode. ASIL confirms dual-signature AuthorRecord coverage for all active	Low-Medium

Phase	Name	Actions	Completion Criteria	Risk Level
		ASIL supervision. ASIL re-signs all existing AuthorRecords using ML-DSA-5 in addition to existing ECDSA-P384 signatures (dual-signature period). Existing AES-256-GCM encryption remains active in parallel. All QTraversalTokens issued henceforth use ML-KEM-1024 encapsulation.	records. QIGEMKeyRecord audit log reviewed by cryptography team.	
Phase 2	ZKALS Enrollment	Issue ZKALSProof verification keys to all VERIFIED+ tier nodes. Run AUTHORSHIP-scope proof exchange for 10 consecutive consensus cycles in shadow mode (proofs generated and verified but not enforced — Consensus Engine continues to use v1.5 plaintext AuthorRecord	10 shadow-mode cycles complete. Proof success rate $\geq 99.5\%$ confirmed. All VERIFIED+ nodes enrolled and generating proofs within the cache refresh interval. ASIL approves enforcement activation.	Medium

Phase	Name	Actions	Completion Criteria	Risk Level
		validation in parallel). Collect ZKALSVerificationResult telemetry. Validate proof success rate $\geq 99.5\%$ across shadow-mode cycles before enforcement activation.		
Phase 3	dAPID Mesh Activation	Compose initial dAPID mesh from SOVEREIGN and VERIFIED tier nodes only (minimum 7 nodes, recommended 13). Run dAPID in parallel with legacy per-node APID for 20 consecutive consensus cycles. Compare per-node APIDReport dispositions against dAPIDRoundResult dispositions. On confirmed agreement rate $\geq 95\%$ across all 20 cycles: emit MeshRecompositionEvent to retire per-node APID.	20 parallel-operation cycles complete. Agreement rate $\geq 95\%$ confirmed. ASIL approves per-node APID retirement. dAPIDRoundResult designated as sole APID signal for Consensus Engine.	Medium-High

Phase	Name	Actions	Completion Criteria	Risk Level
		Apply LEGACY_EVALUATOR classification and 0.5× weight to v1.5 nodes participating in mesh.		
Phase 4	Full v1.6 Operation	Enforce all 8 conditions of the v1.6 Consensus Contract. Enable ZKALSProof as mandatory requirement for all merge operations (shadow mode retired). dAPID designated as sole injection detection mechanism. Apply FTSS penalty schedule to all LEGACY_MODE nodes. Downgrade all non-migrated nodes to PROVISIONAL tier maximum. Activate all 5 new Internal Event Bus event types. Retire all v1.5-specific event consumers and producers where superseded.	Consensus Engine confirmed operating on 8-condition contract. All merge operations producing valid ZKALSPProof and dAPIDRoundR result records. No v1.5 legacy event consumers active for superseded event types. ASIL countersigns Phase 4 completion certification.	High
Phase 5	QIGEM EPOCH_2 (Future)	When all currently active	Unanimous node readiness	High (future)

Phase	Name	Actions	Completion Criteria	Risk Level
		federation nodes individually confirm full PQC readiness: emit ASIL-countersigned EpochAdvancementEvent (EPOCH_1 → EPOCH_2). Retire classical RSA/ECC crypto layer entirely. Retain AES-256-GCM as symmetric cipher. LEGACY_MODULE nodes unable to confirm readiness are suspended from federation prior to EPOCH_2 activation.	confirmation received. ASIL countersignature on EpochAdvancementEvent verified by all nodes. Zero LEGACY_MODULE nodes active in federation. Cryptographic audit confirms complete retirement of RSA/ECC artifacts.	

MIGRATION ROLLBACK POLICY

Phase 4 (Full v1.6 Operation) does not support rollback to v1.5 once the 8-condition Consensus Contract is activated. A rollback-capable staging environment must be maintained through Phases 1-3. ASIL must formally document the point-of-no-return threshold before Phase 4 activation. Any post-Phase-4 issues must be resolved through v1.6 operational procedures, not by reverting to the v1.5 contract. This constraint is by design: reverting to the v1.5 5-condition contract re-opens the threat surfaces that v1.6 was constructed to

close.

Section 10 — One-Line System Definition (v1.6)

CK v1.6 is a quantum-adversarially-hardened federated cognitive governance system that replaces classical cryptographic identity protection with post-quantum CRYSTALS-Kyber/Dilithium encryption (

QIGEM

), eliminates author identity exposure during lineage validation via zk-STARK zero-knowledge proofs (

ZKALS

), and defeats coordinated injection falsification through Byzantine-fault-tolerant distributed APID mesh consensus (

dAPID

) — all operating within the unchanged CK v1.5 governance precedence hierarchy.

Section 11 — Roadmap: CK v1.7 Extensions (Preview)

The following extensions are formally scheduled for CK v1.7. Items previously noted in the CK v1.5 roadmap are so indicated. This section is informational only; no v1.7 feature constitutes an architectural commitment in this specification. All items are subject to formal specification in the CK v1.7 design document.

Extension	Category	Description	v1.5 Roadmap Item
Autonomous Trust Graph Healing	FTSS Enhancement	FTSS self-repair mechanism for minor trust score degradations that fall within a defined recovery threshold, allowing resolution without CIEMS arbitration. Reduces governance overhead for routine trust fluctuations while preserving CIEMS authority for material degradations.	Yes — formally scheduled
Cognitive Load Balancing Protocol	CK Core Enhancement	CK Core workload distribution mechanism enabling verified federation nodes to accept compute delegation for non-sensitive reasoning tasks. Includes trust-gated delegation policy, ASIL oversight of delegation scope, and FTSS scoring of delegation reliability.	Yes — formally scheduled
Homomorphic Reasoning Layer	New Capability	Enables CK Core to perform governance-relevant computations over encrypted inputs without	No — new for v1.7

Extension	Category	Description	v1.5 Roadmap Item
		decryption. Governance logic (CIEMS constraint evaluation, FTSS scoring) would operate on ciphertext representations, eliminating the need to decrypt sensitive inputs for governance processing. Depends on Fully Homomorphic Encryption (FHE) maturity and performance benchmarks.	
Self-Healing dAPID Mesh	dAPID Enhancement	Extension to dAPID that enables the mesh to detect and isolate Byzantine nodes mid-round without requiring a full mesh recomposition. Suspected Byzantine nodes are flagged and their verdicts down-weighted in real time during the current round; forced recomposition occurs after round completion rather than as a blocking operation. Reduces latency impact of Byzantine detection events.	No — new for v1.7
ASIL Temporal Binding	ASIL Enhancement	Cryptographic timestamping protocol that binds AuthorRecord entries to a	No — new for v1.7

Extension	Category	Description	v1.5 Roadmap Item
		verifiable timestamp at the moment of record creation. Provides non-repudiation proof that an AuthorRecord existed (and was ASIL-authenticated) at a specific time, independently of any subsequent ZKALS proof exchange. Relevant for dispute resolution, regulatory compliance, and long-term archival integrity verification.	

Cognitive Kernel v1.6 — Quantum Adversarial Extension Specification Wolf-CoG-OS / AAIS Federated Cognitive Architecture Status: Draft-Stable (Implementation Candidate)	Prepared by: jon Grayling Charter Township, MI, United States 15 June 2026 01:34 EDT Supersedes: CK v1.5
--	---

This document is a draft-stable specification of the Wolf-CoG-OS / AAIS Federated Cognitive Architecture. All architectural definitions, schema specifications, and protocol rules contained herein are authoritative for CK v1.6 conformant implementations. Implementation candidates must satisfy all conditions specified in the v1.6 Consensus Contract (Section 5.3) prior to federation admission.